CS 2160: Computer Organization & Assembly Language Programming

Instructor: Himon Thakur

Fall 2026

Assignment #2

- This assignment is worth **100 points**.
- Please type your submissions and format properly (the TA will thank you)
- **10% bonus points** will be available for anyone submitting LaTeX-formatted homework (code + PDF)
- Please use the internet and all other resources to help you with the assignment, not write it for you

Problem 01 (25 pts)

(a) (5 pts) Convert $(2727)_{10}$ into a **32-bit** *two's complement* binary number.

(b) (5 pts) Convert $(-5795)_{10}$ into a **16-bit** *two's complement* binary number.

(c) (15 pts) **Add** the above two binary numbers together as **32-bit** *two's complement* binary numbers by sign-extension and padding, writing your answer as an **8-hextet hexadecimal** value.

Problem 02 (25 pts)

For this problem, assume we are using **8-bit** registers, and all values are in **two's complement** *signed integer* format.

	`t0`	`t1`
(a)	$(57)_{10}$	$(71)_{10}$
(b)	$(1111\ 0000)_2$	$(1001\ 0000)_2$
(c)	$(0110\ 1000_2)$	$(0011\ 1101)_2$

Suppose we run the instruction

```
add t2 , t0 , t1
```

(10 pts) For each of the cases, did **overflow** occur or not?

(15 pts) What is the **binary value** in register `t2` for all cases?

Problem 03 (25 pts)

(a) (10 pts) What is the **instruction type** and Assembly language **instruction** for the following hexadecimal value?

0x001080B3

(b) (15 pts) Convert the following RISC-V instruction to the **32-bit** binary value of its *machine instruction representation* (hexadecimal) and explain your answer:

```
addi sp , sp , -16
```

Problem 04 (25 pts)

(a) (10 pts) Find the **decimal number** equivalent of the following **IEEE 754** *single-precision* numbers.

i. `1 0101 1011 1001 1010 1100 0000 0000 000`

ii. `0 1000 1110 0100 1010 1010 1010 1010 101`

iii. `0 1111 1111 0001 1000 0101 1010 0100 001`

iv. `0 0000 0000 1011 0000 0000 0000 0000 000`

v. `1 0000 0001 0000 0000 0000 0000 0000 000`

If you find a *repeating pattern*, write the *fraction* you think it is, and round to the **ninth** fractional digit (*billionths*).

(b) (10 pts) Represent the following numbers using **32-bit** *single-precision* floating point format as per the **IEEE 754** standard.

i. $(0.8)_{10}$

ii. $(-22.7)_{10}$

iii. $(5.625)_{10}$

iv. NaN

v. Negative infinity

(c) (5 pts) Suppose you only have **6 digits** for representing the decimal number **0.8**, i.e., $0.b_1b_2b_3b_4b_5b_6$. How far off is this number from the actual value **0.8**? In this problem, do not round the last bit: b_6 is the last bit before truncation of the full representation.